Experiment-3

Aim: To create a database schema for a Customer-Sale scenario, insert records, and perform various SQL queries including joins, aggregation, and views.

Explanation:

- Customer – stores customer details.
- Item – stores product details and price.
- Sale – stores transaction details and connects Customer and Item using foreign keys.

Primary keys were used to uniquely identify records, and foreign keys were used to maintain relationships between tables.

Various SQL queries were performed to:

- Display bills for the current date
- Calculate total bill amount
- Count products purchased by each customer
- List products bought by specific customers
- Create views for bill details and weekly sales

#Code

Tables with Appropriate Integrity Constraints

```sql
CREATE DATABASE customer_sale;

use customer_sale;

CREATE TABLE Customer (
Cust_id INT PRIMARY KEY,
Cust_name VARCHAR(100) NOT NULL
);

CREATE TABLE Item (
    item_id INT PRIMARY KEY,
    item_name VARCHAR(100) NOT NULL,
    price INT NOT NULL CHECK (price>0)
);

CREATE TABLE Sale (
    bill_no INT PRIMARY KEY,
    bill_date DATE NOT NULL,
    cust_id INT,
    item_id INT,
    qty_sold INT NOT NULL CHECK (qty_sold > 0),
    FOREIGN KEY (cust_id) REFERENCES Customer(cust_id),
    FOREIGN KEY (item_id) REFERENCES Item(item_id)
);
```

Insert around 10 records in each table.

```
25 •    INSERT INTO Customer VALUES
26      (1,'Priyanshi'),
27      (2,'Priya'),
28      (3,'Aman'),
29      (4,'Akansha'),
30      (5,'Dolly'),
31      (6,'Neha'),
32      (7,'Rohit'),
33      (8,'Aditya'),
34      (9,'Vishal'),
35      (10, 'Varun');
36
37 •    INSERT INTO Item VALUES
38      (101,'Laptop',50000),
39      (102,'Mouse',500),
40      (103,'Keyboard',1500),
41      (104,'Monitor',12000),
42      (105,'Pen Drive',800),
43      (106,'Headphones',2500),
44      (107,'Speaker',3000),
45      (108,'Notebook',100),
46      (109,'Printer',15000),
47      (110,'Tablet',20000);
```

```sql
INSERT INTO Sale VALUES
(1,CURDATE(),1,101,1),
(2,CURDATE(),2,102,2),
(3,CURDATE(),3,104,1),
(4,CURDATE(),5,106,3),
(5,CURDATE(),5,108,5),
(6,'2026-02-15',4,109,1),
(7,'2026-02-14',6,105,4),
(8,'2026-02-13',7,103,2),
(9,'2026-02-12',8,110,1),
(10,'2026-02-11',9,107,2);

SELECT s.bill_no, c.cust_name, s.item_id
FROM Sale s
JOIN Customer c ON s.cust_id = c.cust_id
WHERE s.bill_date = CURDATE();

SELECT s.bill_no,
       c.cust_name,
       i.item_name,
       s.qty_sold,
       i.price,
       (s.qty_sold * i.price) AS final_amount
```

```sql
SELECT s.bill_no, c.cust_name, s.item_id
FROM Sale s
JOIN Customer c ON s.cust_id = c.cust_id
WHERE s.bill_date = CURDATE();

SELECT s.bill_no,
       c.cust_name,
       i.item_name,
       s.qty_sold,
       i.price,
       (s.qty_sold * i.price) AS final_amount
FROM Sale s
JOIN Customer c ON s.cust_id = c.cust_id
JOIN Item i ON s.item_id = i.item_id;


SELECT DISTINCT c.cust_id, c.cust_name
FROM Sale s
JOIN Customer c ON s.cust_id = c.cust_id
JOIN Item i ON s.item_id = i.item_id
WHERE i.price > 200;
```

```sql
SELECT c.cust_id, c.cust_name,
       SUM(s.qty_sold) AS total_products_bought
FROM Sale s
JOIN Customer c ON s.cust_id = c.cust_id
GROUP BY c.cust_id, c.cust_name;

SELECT i.item_name, s.qty_sold
FROM Sale s
JOIN Item i ON s.item_id = i.item_id
WHERE s.cust_id = 5;

SELECT DISTINCT i.*
FROM Sale s
JOIN Item i ON s.item_id = i.item_id
WHERE s.bill_date = CURDATE();
```

```sql
CREATE VIEW Bill_Details AS
SELECT s.bill_no,
       s.bill_date,
       s.cust_id,
       s.item_id,
       i.price,
       s.qty_sold,
       (i.price * s.qty_sold) AS amount
FROM Sale s
JOIN Item i ON s.item_id = i.item_id;

SELECT * FROM Bill_Details;

CREATE VIEW Weekly_Sales AS
SELECT bill_date,
       SUM(i.price * s.qty_sold) AS total_daily_sales
FROM Sale s
JOIN Item i ON s.item_id = i.item_id
WHERE bill_date >= CURDATE() - INTERVAL 7 DAY
GROUP BY bill_date;

SELECT * FROM Weekly_Sales;
```

#Output:

1. List All Bills for Current Date with Customer Names and Item Numbers

bill_no	cust_name	item_id
1	Priyanshi	101
2	Priya	102
3	Aman	104
4	Dolly	106
5	Dolly	108

2. List Total Bill Details (Quantity, Price, Final Amount)

Result Grid | Filter Rows: | Export: | Wrap Ce

bill_no	cust_name	item_name	qty_sold	price	final_amount
1	Priyanshi	Laptop	1	50000	50000
2	Priya	Mouse	2	500	1000
3	Aman	Monitor	1	12000	12000
4	Dolly	Headphones	3	2500	7500
5	Dolly	Notebook	5	100	500
6	Akansha	Printer	1	15000	15000
7	Neha	Pen Drive	4	800	3200
8	Rohit	Keyboard	2	1500	3000
9	Aditya	Tablet	1	20000	20000
10	Vishal	Speaker	2	3000	6000

3. Customers Who Bought Product with Price > 200

cust_id	cust_name
1	Priyanshi
2	Priya
7	Rohit
3	Aman
6	Neha
5	Dolly
9	Vishal
4	Akansha
8	Aditya

4. Count of Products Bought by Each Customer

Result Grid | Filter Rows:

cust_id	cust_name	total_products_bought
1	Priyanshi	1
2	Priya	2
3	Aman	1
5	Dolly	8
4	Akansha	1
6	Neha	4
7	Rohit	2
8	Aditya	1
9	Vishal	2

5. Products Bought by Customer with cust_id = 5

item_name	qty_sold
Headphones	3
Notebook	5

6. Item Details Sold Today

item_id	item_name	price
101	Laptop	50000
102	Mouse	500
104	Monitor	12000
106	Headphones	2500
108	Notebook	100

7. Create a View for Bill Details

Result Grid | Filter Rows: | Export: | Wrap Cel

bill_no	bill_date	cust_id	item_id	price	qty_sold	amount
1	2026-02-28	1	101	50000	1	50000
2	2026-02-28	2	102	500	2	1000
3	2026-02-28	3	104	12000	1	12000
4	2026-02-28	5	106	2500	3	7500
5	2026-02-28	5	108	100	5	500
6	2026-02-15	4	109	15000	1	15000
7	2026-02-14	6	105	800	4	3200
8	2026-02-13	7	103	1500	2	3000
9	2026-02-12	8	110	20000	1	20000
10	2026-02-11	9	107	3000	2	6000

8. View for Daily Sales (Last One Week)

Result Grid | Filter Rows:

bill_date	total_daily_sales
2026-02-28	71000